

Exercise 1 — Breaking Things

Goal: Find and exploit the real vulnerabilities in the Wallet API. By the end of this exercise you will have accessed another user's data, seen their plain-text password, and understood exactly why it happened.

Setup checklist

- ☐ API is running: `docker compose up` in the main folder (where `docker-compose.yml` lives)
 - ☐ Postman is open with the "API Security Techniques – Wallet API" collection imported
 - ☐ You've logged in as Alice (run "1 – Authentication → Login as Alice" — token auto-saved)
-

The scenario

You are **Alice** (`alice` / `password123`). The API has three other users: **Bob** (`bob`), **Charlie**, and **admin**.

Your job: prove you can access their data using only Alice's token.

Challenge 1 — IDOR on User Profiles

Step 1 — Fetch your own profile

Run: "2 – Exercise 1: Breaking Things → My Profile (Alice – OK)"

Look at the response. Notice anything sensitive?

```
{
  "id": 1,
  "username": "alice",
  "email": "alice@example.com",
  "password": "password123",    // ← What is this doing here?
  "role": "USER"
}
```

Vulnerability: Sensitive data leakage — the password field is returned in the API response.

Step 2 — Read Bob's profile

Run: "🔥 IDOR – Alice reads Bob's profile (id=2)"

You are Alice. You just read Bob's full profile — including his password `hunter2` .

Vulnerability: `GET /api/users/{id}` has no ownership check. Any authenticated user can read any other user's profile by incrementing the ID.

Step 3 — Read the Admin's profile

Run: "🔥 IDOR – Alice reads Admin profile (id=4)"

You now have the admin's password. In a real system, what could you do with that?

Discussion: Why is the controller not the right place to stop this?

Challenge 2 — IDOR on Accounts

Step 1 — Fetch your own accounts (correct)

Run: "My Accounts (Alice – OK)" You see Alice's account: ACC-ALICE-001 , balance 5000.00 . ✓

Step 2 — Enumerate other accounts

Run: "🔥 IDOR – Alice reads Bob's account (id=2)" Run: "🔥 IDOR – Alice reads Admin account (id=4)"

You can see every account's balance by changing the ID.

Vulnerability: GET /api/accounts/{id} in the service layer performs no ownership check. The controller passes the raw {id} path variable directly through.

Look at the code:

```
// AccountService.java – the bug is here, not in the controller
public Account getAccountById(Long id) {
    return accountRepository.findById(id)           // no "does this belong to the
    caller?" check
        .orElseThrow(() -> new RuntimeException("Account not found"));
}
```

Challenge 3 — IDOR on Transactions

Step 1 — Read your own transactions

Run: "Alice's Transactions (account 1 – OK)"

Step 2 — Read other users' transactions

Run: "🔥 IDOR – Alice reads Bob's transactions (account 2)" Run: "🔥 IDOR – Alice reads Admin transactions (account 4)"

Same pattern — the accountId path variable is trusted without verification.

Discussion: There is also a search endpoint — GET /api/transactions/account/{accountId}/search?keyword=. It has the same missing ownership check. Every new feature built on top of the service inherits the vulnerability. This is why the fix belongs in the service layer: fix it once, every endpoint is protected.

Challenge 4 — Brute-force Login

Step 1 — Try wrong passwords

Run "🔥 Brute-force Login (wrong passwords)" 10 times in a row — rapidly.

Notice: the API returns 500 every time. No lockout. No delay. No alert.

Step 2 — Calculate the risk

A 6-character lowercase password = $26^6 = \sim 308$ million combinations. At 10 requests/second (easily achievable with Postman Runner or curl):

- $308,000,000 / 10 = 30,800,000 \text{ seconds} = \sim 1 \text{ year}$ at 10 req/s
- At 1,000 req/s (a cheap VPS): **3.5 days**
- With a dictionary of 10,000 common passwords: **10 seconds**

Vulnerability: No rate limiting on `POST /api/auth/login`.

Challenge 5 — Fake Webhook

Step 1 — Send a forged payment event as Admin (id=4)

Run: "3 – Exercise 2 → 🔥 Fake Webhook (no signature)"

The API responds `200 OK` and logs:

```
Received webhook event: payment.completed for account 4
```

Vulnerability: The `X-Signature-256` header is received but never validated. Any external system can trigger internal payment events.

Challenge 6 — Exposed Actuator Endpoints ⌚ if time

Step 1 — Hit the metrics endpoint with no token

Run: "🔥 Exposed Actuator – Metrics (no auth needed)"

You get a full list of available metrics — no `Authorization` header required.

Run: "🔥 Exposed Actuator – JVM Memory (no auth needed)"

```
{ "name": "jvm.memory.used", "measurements": [{ "statistic": "VALUE", "value": 123456789 }] }
```

Run: "🔥 Exposed Actuator – Info (no auth needed)"

Vulnerability: `/actuator/**` is in the `permitAll()` list in `SecurityConfig`. An attacker can fingerprint the runtime (JVM version, memory, thread counts) without credentials — useful intelligence before a targeted attack.

```
// SecurityConfig.java
.requestMatchers("/actuator/**").permitAll() // ← no auth required
```

Challenge 7 — JWT Inspection ⌚ if time

No Postman request needed — do this manually.

Step 1 — Copy Alice's token

After logging in, copy the value of `{{alice_token}}` from the collection variables panel.

Step 2 — Decode it at jwt.io

Go to jwt.io and paste the token into the Encoded field.

The payload decodes immediately — no secret needed to read it:

```
{
  "sub": "alice",
  "role": "USER",
  "iat": 1700000000,
  "exp": 1700086400
}
```

Step 3 — Crack the signature

In the **Verify Signature** box, type `secret123` .

The signature turns valid  — the token is fully compromised.

Vulnerability: The JWT secret (`secret123`) is 9 characters, hardcoded in `application.yml` , and trivially brute-forced offline. Anyone who intercepts a token can forge new ones with any `role` they like — including `ADMIN` .

Bonus — H2 Database Console (browser only)

The API ships with an in-memory H2 database and its web console exposed with no authentication.

Step 1 — Open the console in your browser

Go to: <http://localhost:8080/h2-console>

No token. No login prompt. It just opens.

Step 2 — Connect to the database

Use these settings:

- **JDBC URL:** `jdbc:h2:mem:walletdb`
- **Username:** `sa`
- **Password:** *(leave blank)*

Click **Connect**.

Step 3 — Dump every password

```
SELECT * FROM app_user;
```

You now have every username, email, and plain-text password in the system in one query.

Vulnerability: Same root cause as the actuator — `permitAll()` on internal tooling that should never be reachable in production:

```
// SecurityConfig.java
.requestMatchers("/h2-console/*").permitAll() // ← full DB access, no auth
```

Discussion: H2 console should be disabled entirely in production via `application.yml` :

```
spring:
  h2:
    console:
      enabled: false
```

Debrief — What did we break?

Vulnerability	Root cause
IDOR – User profiles	No ownership check in service layer
IDOR – Accounts	No ownership check in service layer
IDOR – Transactions	No ownership check in service layer
Sensitive data leak	Password field in response DTO
Brute-force login	No rate limiting on auth endpoint
Fake webhook	No HMAC signature verification
Exposed actuator	permitAll() on /actuator/**
Weak JWT secret	9-char hardcoded secret, crackable offline
H2 console exposed	permitAll() on /h2-console/** — full DB via browser

"Everything you just broke is because one of the 3 fundamentals was missing: auth/authorization, input validation, or encryption."

Head to the break — Exercise 2 is where you fix all of this.